FR

Bienvenue sur Sakata

Ce portail est un sanctuaire numérique dédié à la préservation et à la transmission de l'identité culturelle du peuple **Sakata**.

Notre objectif est d'unir nos racines ancestrales avec la splendeur numérique moderne, en offrant à la fois des archives vivantes, des sagesses intemporelles et un forum d'échange intergénérationnel.

Suivant →

> *Sans contrôle, un composant React monté simultanément avec d'autres peut lancer 20 requêtes Supabase en parallèle. Les sockets se saturent, l'auth timeout, le chat se déconnecte. Le proxy de traffic control est la solution maison pour garantir que les sockets prioritaires (auth, profils) sont toujours dispos.*

PROBLÈME

Pourquoi

Supabase Postgrest tourne sur HTTP/1.1 par défaut côté navigateur. La plupart des navigateurs limitent à 6 connexions parallèles par origine. Si on en consomme 6 simultanément pour des SELECT lourds, plus aucune n'est dispo pour :

- Refresh du JWT auth (bloque l'utilisateur).
- Heartbeat WebSocket Realtime (chat se croit déconnecté).
- Chargement des assets statiques (avatars, images).

SOLUTION

Limiter à **4 requêtes Postgrest simultanées**, garder 2 sockets libres pour Auth et assets. File d'attente FIFO avec priorisation des appels critiques.

CODE

Implémentation

Le proxy enveloppant le client

SRC/LIB/SUPABASE.TS (EXTRAIT) TYPESCRIPT

```typescript
const MAX_CONCURRENT = 4;
let activeRequests = 0;
const requestQueue: QueueEntry[] = [];

async function acquireRequest(callName: string, isPriority = false) {
  if (activeRequests < MAX_CONCURRENT) {
    activeRequests++;
    return;
  }
  return new Promise<void>(resolve => {
    if (requestQueue.length > 5) {
      console.warn(`[NET-SATURATION] File d'attente: ${requestQueue.length}`);
    }
    requestQueue.push({ resolve, callName, isPriority });
  });
}

function releaseRequest() {
  activeRequests--;
  if (requestQueue.length > 0) {
    // Priorité aux requêtes critiques
    const nextIndex = requestQueue.findIndex(q => q.isPriority);
    const next = nextIndex !== -1
      ? requestQueue.splice(nextIndex, 1)[0]
      : requestQueue.shift();

    if (next) {
      activeRequests++;
      next.resolve();
    }
  }
}
```

Priorisation des appels profile

Un appel `from("profiles")` est marqué automatiquement comme prioritaire — c'est ce dont l'auth dépend pour vérifier les rôles.

CODE TYPESCRIPT

```typescript
if (prop === "then") {
  return (onfulfilled, onrejected) => {
    const isPriority = callName.includes("profiles");

    return acquireRequest(callName, isPriority).then(() => {
      // ... exécuter la requête
    });
  };
}
```

OBSERVABILITÉ

Logs structurés

TAG	SENS
`[NET-QUEUE-WAIT]`	Une requête a dû attendre dans la file
`[NET-QUEUE-EXEC]`	Une requête de la file s'exécute
`[NET-START]`	Une requête démarre directement (slot libre)
`[NET-RESOLVED]`	Une requête a abouti (succès ou erreur)
`[NET-SATURATION]`	File > 5 → alerte, possible bug
`[NET-ERROR]`	Requête en erreur, avec callName + ID

DEBUG RAPIDE

Filter Chrome DevTools sur `[NET-` donne immédiatement une vue waterfall de toutes les requêtes Supabase et leur état.

RÈGLE ABSOLUE

Obligation : AbortController

SANS ABORTCONTROLLER, LEAK GARANTI

Quand un composant se démonte, ses requêtes en cours **doivent être abandonnées**. Sinon elles consomment un slot du proxy après le démontage.

CODE TYPESCRIPT

```typescript
useEffect(() => {
  const controller = new AbortController();

  async function fetch() {
    const { data, error } = await supabase
      .from("articles")
      .select()
      .abortSignal(controller.signal);

    if (controller.signal.aborted) return;
    setArticles(data);
  }

  fetch();

  return () => controller.abort();  // ← OBLIGATOIRE
}, []);
```

À CONNAÎTRE

Cas limites

Realtime (WebSocket) hors proxy

Le proxy n'intercepte que `from()` et `rpc()`. Les channels Realtime utilisent un WebSocket dédié et ne consomment pas de slot HTTP — c'est intentionnel.

Storage hors proxy

Les uploads/downloads via `supabase.storage` ne passent pas par le proxy. Pour les gros fichiers (vidéo héro), c'est correct — ils ont leurs propres limites de bande passante.

CALIBRAGE

Tuning de MAX_CONCURRENT

Pourquoi 4 et pas 6 ? Compromis empirique :

- **2 :** trop conservateur, file s'accumule en burst.
- **4 :** sweet spot — garantit auth/assets dispos.
- **6 :** sature, l'auth peut timeout en burst.
- **8+ :** dépasse la limite navigateur, requêtes en attente même sans proxy.

À SAVOIR

Limitations & évolutions

- Le proxy est in-memory : si on a plusieurs onglets, chacun a sa propre file.
- Pas de circuit breaker — un Supabase down sature la file indéfiniment (timeout fallback à ajouter).
- HTTP/2 sur Supabase pourrait rendre le proxy moins critique (à monitorer).

« Moto oyo ayebi te kotombola miso na likolo, akoki te koyeba esika oyo mbula eutaka. » — Proverbe Sakata

SAKATA

© 2026 — Brume de la Rivière. Tous droits réservés.

CULTURE ECOLE LANGUE HISTOIRE FORUM